

Exercise — Ship From Slack

Wire Slack, Notion and GitHub to your project with the orchestration harness

Repo: `Orenda-Project/orchestration-harness-v2` · about 3 hours · do it after videos 1–8.

You finish when a message you type in Slack becomes a Notion ticket, an agent session, a pull request on GitHub, and a reply in your thread — without opening your editor.

Work only in your own fork, against a throwaway Notion database, Slack channel and repo.

Module 14 Exercise — Ship From Slack

What you are building: a working loop where you type a message in Slack, and your project changes — without opening your laptop, your editor, or Claude Code. Slack is the front door, Notion is the work board, GitHub is where the change lands, and the **orchestration harness** is the thing in the middle that keeps checking all three and putting an agent to work.

The test of done — one sentence: *from your phone, in Slack, you ask for a change to your project, and within three minutes a Notion ticket exists, an agent has worked on it, a pull request is open on GitHub, and the agent has replied in your Slack thread.*

Time: about 3 hours, in one sitting if you can. **Difficulty:** you have finished videos 1–8 of this module. **You will use:** the repo `Orenda-Project/orchestration-harness-v2`.

Before you start

Tick every box before you clone anything. Missing one of these is the most common reason this exercise takes five hours instead of three.

- ☐ You have watched videos 3, 7 and 8 of this module, and your Notion integration and Slack app already exist.
- ☐ You can install a Slack app in your workspace — **or** you have asked an admin and they have said yes. Ask today, not on the day you start.
- ☐ You have access to the harness repo. **It is private** — your instructor will grant you access or give you a copy. You cannot clone it without that.
- ☐ You have a **small project repository of your own** on GitHub that you are willing to let an agent change. Not your dissertation. A scratch repo with a README is perfect.
- ☐ You have `git`, `python3`, `sqlite3` and Claude Code installed and working.
- ☐ You have created **three throwaway things**: a Notion database, a private Slack channel, and a GitHub repo. Nothing real. You will point the harness at these, not at your team's workspace.

Safety brief — read this before you clone. This repo was built for one person's real workspace, and it still has their fingerprints in it. At the time of writing, `harness/tick.sh` contains credentials pasted directly into the script, and the Slack poller is hardcoded to one specific person and one specific bot ID.

Do not run `harness/tick.sh` as it is, do not copy any key you find inside the repo, and do not push anything back to the original repo. Work in your own fork. If any key in that file is still live, it belongs to someone else — report it to your instructor rather than using it. Video 6 of this module is about exactly this failure, and you are about to see it in the wild.

How the harness actually works

One page, so you are not running magic. Read it once now; it will save you an hour later.

The harness has **no server and no daemon**. It is a loop running inside Claude Code. Every **tick** — by default every 180 seconds — it does the same five things:

Step	What happens
1 · sync-state	Takes a tick lock so two ticks can never run at once, and reads <code>last_sync_at</code> — the timestamp it last caught up to.
2 · poll	Asks Notion, Slack and GitHub: <i>what changed since</i> <code>Last_sync_at</code> ? Anything actionable becomes an event row in a local SQLite database.
3 · dispatch	Groups pending events by context key , skips any context that already has a session running, and spawns one agent session per context.
4 · sync-state	Moves <code>last_sync_at</code> forward and releases the lock.
5 · self-improve	Only if the harness is idle and 24 hours have passed. Ignore it for this exercise.

Four words you need, from the repo's own glossary:

- **Event** — one normalised signal ("someone mentioned the agent in Slack"). Its ID is built from `source + external_id`, so the same message can never be queued twice.
- **Entity** — one external object the harness is tracking: a Notion ticket, a Slack thread, a GitHub PR.
- **Context key** — *always the Notion ticket*. Never the Slack thread, never the PR. This is why work that starts in Slack gets a stub Notion ticket created for it automatically.
- **Workspace** — `harness/workspace/`, a local clone of **your** project. This is what the agent edits. It is not the harness repo.

So the shape of the whole thing is: **Slack is the doorbell, Notion is the work board, GitHub is the result, and the loop is the thing that keeps looking.**

Part 1 · Your three doors (25 min)

You need three credentials. Two of them you already made in videos 7 and 8.

- ☐ **Notion.** Your integration's secret, your database ID, and the agent's Notion user ID. Share the database with the integration under `...` → **Connections** — creating it grants nothing.
- ☐ **Slack.** Your bot token (`xoxb-`), your test channel's ID (`C0...`), and the bot invited to that channel with `/invite`.
- ☐ **Slack, one extra thing.** The harness's Slack poller uses the **search** API, and search needs a **user token** (`xoxp-`) with the `search:read` scope — a bot token cannot search. Add it, reinstall, and keep it beside the bot token.
- ☐ **GitHub.** A personal access token with repo scope, and your target repo as `owner/repo`.

Least privilege, as taught in video 3. Scope each of these to the throwaway resources only. The GitHub token should reach your scratch repo and nothing else you care about.

Evidence E1: a screenshot of your Slack app's Bot Token Scopes and your User Token Scopes, with the token values themselves *not* visible.

Part 2 · Fork, clone, and run setup (30 min)

- ☐ Fork the harness into your own account, then clone your fork. Never work in the original.
- ☐ Open Claude Code **inside the cloned harness folder**.
- ☐ Run `/setup`. It is a wizard skill — it will walk you through its phases and stop at the first failure with a remediation message.

The wizard will ask for every variable in `harness/.env.example` and then **validate each one before writing the file**. That validation is the useful part: it is the difference between "I pasted a token" and "the token works". It checks, in order:

Service	Check	Pass looks like
Notion	<code>GET /v1/users/me</code>	HTTP 200
Notion	<code>GET /v1/databases/<your db id></code>	HTTP 200 — a 404 here means you skipped Connections
Notion	<code>GET /v1/users/<agent user id></code>	HTTP 200
Slack	<code>POST auth.test</code>	<code>ok</code>
Slack	<code>conversations.info</code> for your channel	<code>ok</code> — <code>not_in_channel</code> means you skipped <code>/invite</code>
GitHub	<code>GET /repos/<owner>/<repo></code>	HTTP 200

- ☐ Confirm `harness/.env` now exists, and confirm it is **not** tracked by git: run `git status` and check that `.env` does not appear. The repo's `.gitignore` already excludes it.

Evidence E2: your terminal output showing all six validations passing, with token values masked.

Part 3 · Provision the Notion board (20 min)

The harness expects your Notion database to have four custom properties beyond `Status`:

`Agent Session ID` · `Last Agent Update` · `GitHub PR` · `Slack Thread`

- ☐ Let the setup wizard's provisioning phase add them, or add them by hand in Notion.
- ☐ **Find the title-property trap.** The repo's own skills disagree about what your title column is called — one place expects `Task name`, another expects `Deliverable`. Open your database, find what **your** title property is actually called, and make the code you run agree with it. Write down which files you had to change.

This is the exercise, not a bug to route around. Real integration work is mostly making two systems agree about names. If you paper over it, the poller will create tickets that are invisible to the dispatcher and you will spend an hour wondering why nothing runs.

Evidence E3: a screenshot of your Notion database showing all five properties, and one sentence naming the file(s) you edited for the title property.

Part 4 · The first tick (20 min)

- ☐ Run a single tick and watch it. Do not start the full loop yet.
- ☐ Read the database directly and prove to yourself what state it holds:

```
sqlite3 harness/db/harness.db "SELECT FROM sync_state;" sqlite3 harness/db/harness.db "SELECT FROM tick_lock;" sqlite3 harness/db/harness.db "SELECT id, source, type, status FROM events;"
```

- ☐ Expect **zero events** on a fresh harness. That is a pass, not a failure — `last_sync_at` has only just been set, and nothing has happened since.
- ☐ Confirm the tick released its lock. A tick that crashes mid-way leaves the lock held, and the next tick will refuse to run for 30 minutes.

Evidence E4: the output of those three queries after your first successful tick.

Part 5 · Order work from Slack (40 min)

This is the part the whole exercise exists for.

- ☐ Find your **bot's Slack user ID** (`U...`), and point the Slack poller at it. As shipped, the poller searches for a specific person's bot from another workspace — it will never match yours until you change it.
- ☐ While you are in that file, notice that it also filters for one named person's messages. Decide what your rule should be, and change it. Write down what you chose.
- ☐ Start the loop.
- ☐ In your test channel, post a message that mentions your bot and asks for something small and real. For example: "*@your-bot please add a CONTRIBUTING.md to the repo with a one-paragraph description of the project.*"
- ☐ Wait one tick — up to 180 seconds. Do not touch anything. This is the hard part.

Then check, in this order, that each link in the chain happened:

Check	Where	What you should see
The event was queued	SQLite <code>events</code> table	a row with source <code>slack</code> , status <code>pending</code> then <code>done</code>
A ticket was created	Your Notion board	a new row titled <code>From Slack: <timestamp></code> , status Not Started, with the Slack thread URL filled in
A session was dispatched	SQLite <code>sessions</code> table	one row, status moving <code>scheduled → running → completed</code>
The agent worked	<code>harness/workspace/</code>	your project, checked out locally, with the change made
It spoke back	Your Slack thread	a reply from the bot in the same thread, not a new channel message

If nothing happened, do not restart everything. Work the chain in order and find the first broken link. Nine times out of ten it is one of four things: the bot ID you did not change, `last_sync_at` being newer than your message, the title property name, or a held tick lock.

Evidence E5: a screenshot of your Slack thread showing your request and the bot's reply, and a screenshot of the auto-created Notion ticket.

Part 6 · Make it change the project (30 min)

A reply in Slack is not a shipped change. Now close the loop to GitHub.

- ☐ From Slack, ask for a change that must end up as a **pull request** on your scratch repo.
- ☐ Confirm the PR exists on GitHub, and that the Notion ticket's `GitHub PR` property points at it.
- ☐ Reply in the Slack thread asking for one revision to that PR, and confirm the same context key is reused — you should get **one** session continuing the work, not a second ticket.
- ☐ Merge it yourself. The agent proposes; you approve. That is video 6's rule and it still applies here.

Evidence E6: the PR URL, and a screenshot of the Notion ticket showing the PR link and a status that moved.

Part 7 · Break it on purpose (25 min)

You have met these four errors in videos 3, 5 and 7. Now cause each one deliberately, write down the exact message, and fix it. An integration you cannot break on demand is one you cannot debug under pressure.

#	Do this	Expect	Then fix it by
1	Remove your integration from the Notion database's Connections	<code>object_not_found</code> (404)	re-adding it under Connections
2	Remove the bot from your Slack channel	<code>not_in_channel</code>	<code>/invite @your-bot</code>
3	Delete a scope from your Slack app without reinstalling	<code>missing_scope</code>	adding it back and reinstalling
4	Start a tick, kill it mid-way, then start another	the second tick refuses — the lock is still held	clearing the stale lock, or waiting 30 minutes

Evidence E7: a four-row table, in your own words, of what you did, what the error said, and what fixed it.

Part 8 · Make it safe (20 min)

- ☐ Confirm `.env` is in `.gitignore` and that `git status` never shows it.
- ☐ **Scan your own fork for secrets before you push anything.** Search the whole repo for the token prefixes you now recognise — `ntn_`, `xoxb-`, `xoxp-`, `xapp-`, `ghp_`. Anything you find that is not in `.env` is a problem, whoever wrote it.
- ☐ Write down, in one line each: which action your agent must **never** take without you approving it first, and where you would look to find out what it did.
- ☐ Rotate anything you leaked during this exercise. Assume a key that touched a chat window is burned.

Evidence E8: the output of your secret scan on your fork, and your two one-line answers.

Part 9 · Read the repo critically (20 min)

This repo works, and it is also full of decisions you should not copy. That is normal — most real internal tooling looks like this.

- ☐ Find **three places** where this repository contradicts something this module taught you. For each one, write: what the repo does, which video says otherwise, and what you would change.
- ☐ At least one of your three must be about **credentials**.
- ☐ Look at the `Notion-Version` header the repo sends, then look at what video 4 said about database IDs and data sources. Write two sentences on what would break, and when.

Evidence E9: your three findings, in a short table.

What to hand in

One document (PDF or markdown), named `module-14-<yourname>.md`, containing:

- The link to **your fork** of the harness, and to **your scratch project repo**.
- Evidence **E1 through E9**, in order, each with a one-line caption saying what it proves.
- A short section titled **"What broke and how I found it"** — the real story, including the thing that cost you the most time. This is marked, and honesty scores higher than a clean run.
- Two sentences: what you would let this harness do on your real project, and what you would not.

Mask every token in every screenshot. A submission containing a live credential is returned unmarked, and you should rotate that credential immediately.

How this is marked (100 points)

Area	Points	What earns full marks
Setup validated	15	All six service validations pass (E2), <code>.env</code> untracked
Notion board correct	10	Five properties present, title-property mismatch found and fixed, files named (E3)
First tick clean	10	Lock acquired and released, <code>last_sync_at</code> moved, queries shown (E4)
Slack → Notion → agent	25	The full chain evidenced (E5): event, stub ticket, session, workspace change, threaded reply
Change lands in GitHub	15	PR exists, linked on the ticket, revision reused the same context key (E6)
Broke it and fixed it	10	Four errors reproduced with exact messages and fixes (E7)
Made it safe	10	Secret scan run on the fork, approval rule and log location named (E8)
Read it critically	10	Three real contradictions, one about credentials, plus the API-version answer (E9)
Deduction	−100	A live credential visible anywhere in the submission

Pass: 60. **Strong:** 80+.

Troubleshooting

Symptom	Most likely cause	Fix
"Tick already running"	A previous tick died holding the lock	Inspect <code>tick_lock</code> ; clear it or wait 30 minutes
No events, ever	Your message is older than <code>last_sync_at</code>	Post a fresh message and wait one full tick
No events, and the message is new	The poller is still searching for the original author's bot ID	Change it to your bot's <code>U...</code> ID
Slack search returns nothing	You are using the bot token for search	Search needs the user token with <code>search:read</code>
Notion 404 on a database you can see	The database was never shared with the integration	<code>...</code> → Connections → add it. Check the parent page too
Ticket created, but nothing dispatches	Title property name mismatch	Make the poller and your database agree
The agent edits the wrong files	You are looking at the harness repo, not <code>harness/workspace/</code>	The workspace is the clone of <i>your</i> project
Two tickets for one conversation	The context key was not reused	Reply in the thread , not as a new message

If you want to go further

- Set the loop interval deliberately and justify it. The repo uses 180 seconds; more frequent polling costs more API calls and more money. What is the right number for a project nobody is watching at 3 a.m.?
- Add a **human approval step** before the agent is allowed to merge anything, and prove it blocks.
- Add one new source. The repo already polls Gmail in places — read that code and decide whether you would ship it.
- Replace polling with an event subscription for one source, and write a paragraph on what you gained and what you now have to run.